

Student: Aljeda Cepele

Instructor: Dong Jae Kim

Course: Software Testing

Date: 06/08/2026

Final Project Reflection Report

This project gave me the chance to learn how artificial intelligence can be used in software development and testing. Throughout the quarter, I worked with MCP servers, GitHub tools, automated testing, and an LLM-powered chatbot. Before this class, I had never used MCP or built an application that communicates with a large language model, so this project was a completely new experience for me.

The project was divided into three sprints. Each sprint built on the previous one and helped me understand how AI can be used as a development and testing assistant. By the end of the project, I had a working chatbot application, an MCP server, automated tests, and documentation explaining my testing strategy.

In Sprint 1, I created an MCP server using FastMCP and connected it to VS Code. I created a simple tool that adds two numbers together and verified that VS Code could call the tool successfully through chat. I also created a tester prompt that showed how an AI agent could interact with MCP tools and perform testing-related tasks.

In Sprint 2, I built a Spring Boot chatbot that connects to OpenRouter and sends prompts to a language model. I wanted to add something beyond the starter project, so I created a safety

filter that blocks unsafe prompts before they are sent to the model. I also wrote unit tests for the safety filter and used JaCoCo to generate coverage reports.

In Sprint 3, I focused on documentation, testing strategy, and preparing the final presentation. I updated the README, created an LLM testing strategy document, generated coverage reports, and recorded a demonstration video showing how the system works.

AI played a major role during this project. I used ChatGPT to help me understand concepts, troubleshoot errors, improve code, and create documentation. It helped me work faster, especially when I ran into configuration issues or needed help understanding error messages.

However, I learned that AI is not always correct. There were times when suggested solutions did not work or needed adjustments. Because of this, I always tested the code myself before using it in the project. This project taught me that AI is a useful tool, but developers still need to review and verify everything carefully.

One thing I learned is that testing an AI application is different from testing a normal program.

For the safety filter, I could write traditional unit tests because the expected results were clear. I created tests to make sure harmful prompts were blocked and safe prompts were allowed. Running these tests with Maven helped verify that the feature worked correctly.

For the chatbot itself, testing is more difficult because language model responses can vary. Instead of checking for one exact answer, I focused on evaluating things such as safety,

usefulness, and reliability. I documented these ideas in my LLM testing strategy document and explained how AI systems need additional evaluation methods beyond normal unit testing.

The biggest challenge was getting all the different technologies to work together. The project involved Spring Boot, OpenRouter, FastMCP, GitHub tools, Maven, JaCoCo, and VS Code integration. Sometimes it was difficult to determine whether a problem came from the code, the configuration, or an external service.

One issue I spent time solving was an authentication error when connecting to OpenRouter. The chatbot kept returning authorization errors until I correctly configured the API key. Although it was frustrating, it helped me understand how environment variables work.

Another challenge was understanding code coverage reports. At first, I was unfamiliar with JaCoCo and coverage metrics. After experimenting with it, I learned how coverage reports can help identify parts of the application that still need testing.

One design decision I am happy with was creating the safety filter as a separate service. This made the code easier to organize and easier to test because the filtering logic was isolated from the chatbot logic.

I also organized the application into controllers, services, models, and tests. Keeping the project structured made it easier to find files and understand how everything worked together.

Another decision was using OpenRouter for the chatbot. This allowed the application to connect to language models through a simple API and gave flexibility for future improvements.

Several parts of the project worked successfully.

The MCP server connected correctly to VS Code and was able to run tools through chat. The chatbot successfully communicated with OpenRouter and generated responses. The safety filter worked as expected and prevented unsafe prompts from being processed. My unit tests passed successfully, and JaCoCo generated coverage reports that provided measurable testing results.

I was also happy with the Git workflow because it clearly showed the progress of the project through multiple commits and branches.

If I continued working on this project, I would focus on improving test coverage and creating more automated testing for the chatbot.

I would also like to make the testing agent smarter by having it automatically analyze coverage reports, generate additional tests, and continue improving the test suite without requiring as much manual work.

Another improvement would be adding features such as conversation history, memory, and more advanced safety checks.

This project taught me a lot about AI-assisted software development. I learned how MCP servers work, how AI tools can be integrated into development workflows, and how testing AI systems requires different approaches compared to traditional software.

The biggest lesson I learned is that AI can make development faster, but it does not replace careful testing and verification. Developers still need to understand the code, review results, and make decisions about what should be implemented.

Overall, I enjoyed working on this project because it combined software development, testing, automation, and artificial intelligence into one experience. Building an MCP server, creating a chatbot, writing tests, and exploring AI evaluation strategies helped me gain practical experience with tools and technologies that are becoming increasingly important in the software industry.

This project improved both my technical skills and my understanding of how AI can be used responsibly in software engineering. It was a challenging project, but it gave me valuable experience that I can apply in future courses and professional work.